

```
> ## Documentation Index
> Fetch the complete documentation index at: https://code.claude.com/docs/llms.txt
> Use this file to discover all available pages before exploring further.
```

Best practices for Claude Code

> Suggerimenti e modelli per ottenere il massimo da Claude Code, dalla configurazione dell'ambiente al ridimensionamento tra sessioni parallele.

Claude Code è un ambiente di codifica agenziale. A differenza di un chatbot che risponde alle domande e aspetta, Claude Code può leggere i vostri file, eseguire comandi, apportare modifiche e lavorare autonomamente attraverso i problemi mentre voi guardate, reindirizzate o vi allontanate completamente.

Questo cambia il modo in cui lavorate. Invece di scrivere il codice voi stessi e chiedere a Claude di rivederlo, descrivete quello che volete e Claude capisce come costruirlo. Claude esplora, pianifica e implementa.

Ma questa autonomia comporta comunque una curva di apprendimento. Claude lavora all'interno di determinati vincoli che dovete comprendere.

Questa guida copre i modelli che si sono dimostrati efficaci nei team interni di Anthropic e per gli ingegneri che utilizzano Claude Code in vari codebase, linguaggi e ambienti. Per sapere come funziona il ciclo agenziale sotto il cofano, consultate [How Claude Code works](/it/how-claude-code-works).

La maggior parte delle best practice si basa su un vincolo: la finestra di contesto di Claude si riempie rapidamente e le prestazioni si degradano man mano che si riempie.

La finestra di contesto di Claude contiene l'intera conversazione, inclusi ogni messaggio, ogni file che Claude legge e ogni output di comando. Tuttavia, può riempirsi rapidamente. Una singola sessione di debug o esplorazione del codebase potrebbe generare e consumare decine di migliaia di token.

Questo è importante perché le prestazioni dell'LLM si degradano man mano che il contesto si riempie. Quando la finestra di contesto sta per riempirsi, Claude potrebbe iniziare a "dimenticare" le istruzioni precedenti o fare più errori. La finestra di contesto è la risorsa più importante da gestire. Per vedere come una sessione si riempie nella pratica, [guardate una procedura dettagliata interattiva](/it/context-window) di quello che si carica all'avvio e quanto costa ogni lettura di file. Monitorate continuamente l'utilizzo del contesto con una [custom status line](/it/statusline) e consultate [Reduce token usage](/it/costs#reduce-token-usage) per strategie su come ridurre l'utilizzo dei token.

```
<h2 id="give-claude-a-way-to-verify-its-work">
  Fornite a Claude un modo per verificare il suo lavoro
</h2>
```

```
<Tip>
```

Fornite a Claude un controllo che possa eseguire: test, una build, uno screenshot da confrontare. È la differenza tra una sessione che osservate e una da cui potete allontanarvi.

```
</Tip>
```

Claude si ferma quando il lavoro sembra completato. Senza un controllo che possa eseguire, "sembra completato" è l'unico segnale disponibile, e voi diventate il ciclo di verifica: ogni errore vi aspetta per essere notato. Fornite a Claude qualcosa che produce un risultato di successo o fallimento, e il ciclo si chiude da solo. Claude esegue il lavoro, esegue il controllo, legge il risultato e itera fino a quando il controllo non passa.

Il controllo è qualsiasi cosa che restituisca un segnale che Claude possa leggere nella conversazione: una suite di test, un codice di uscita della build, un linter, uno script che confronta l'output rispetto a un fixture, o uno [screenshot del browser](/it/chrome) confrontato con un design.

Strategia	Prima
Dopo	
----- -----	
----- -----	
----- -----	
Fornire criteri di verifica	*implementare una
funzione che convalida gli indirizzi email"* *"scrivere una funzione validateEmail.	
esempi di casi di test: user@example.com è true, invalid è	
false, user@.com è false. eseguire i test dopo l'implementazione"*	
Verificare visivamente i cambiamenti dell'interfaccia utente	*"rendere il
dashboard più bello"*	*"\[incollare screenshot]
implementare questo design. fare uno screenshot del risultato e confrontarlo con	
l'originale. elencare le differenze e correggerle"*	
Affrontare le cause radici, non i sintomi	*"la build sta
fallendo"*	*"la build fallisce con questo errore:
\[incollare errore]. correggerlo e verificare che la build abbia successo. affrontare la	
causa radice, non sopprimere l'errore"*	

Una volta che il controllo esiste, decidete quanto rigidamente deve bloccare l'arresto:

```
* **In un unico prompt**: chiedete a Claude di eseguire il controllo e iterare nello stesso messaggio, come nella tabella sopra.
```

```
* **In una sessione**: impostate il controllo come condizione [/goal](/it/goal). Un valutatore separato lo ri-controlla dopo ogni turno e Claude continua a lavorare fino a quando non è soddisfatto.
```

```
* **Come un gate deterministico**: un [hook Stop](/it/hooks#stop) esegue il vostro controllo come uno script e blocca il turno dall'terminare fino a quando non passa. Claude Code ignora l'hook e termina il turno dopo 8 blocchi consecutivi.
```

```
* **Da una seconda opinione**: un [subagente di verifica](/it/sub-agents) o un [flusso di lavoro dinamico](/it/workflows) che controlla i propri risultati ha un modello fresco che tenta di confutare il risultato, quindi l'agente che esegue il lavoro non è quello che lo valuta.
```

Ogni passaggio scambia la configurazione per l'attenzione. La versione del prompt funziona su qualsiasi attività oggi. Le versioni `/goal`` e `Stop hook` sono quelle che permettono a un'esecuzione incustodita di terminare correttamente senza di voi.

Fate in modo che Claude mostri prove piuttosto che asserire il successo: l'output del test, il comando che ha eseguito e cosa ha restituito, o uno screenshot del risultato. Rivedere le prove è più veloce che eseguire nuovamente la verifica voi stessi, e funziona per le sessioni che non stavate osservando.

Esplorate prima, poi pianificate, poi codificate

```
<Tip>
  Separate la ricerca e la pianificazione dall'implementazione per evitare di risolvere
il problema sbagliato.
</Tip>
```

Lasciare che Claude salti direttamente alla codifica può produrre codice che risolve il problema sbagliato. Utilizzate [Plan Mode](/it/permission-modes#analyze-before-you-edit-with-plan-mode) per separare l'esplorazione dall'esecuzione.

Il flusso di lavoro consigliato ha quattro fasi:

```
<Steps>
  <Step title="Esplora">
    Entra in Plan Mode. Claude legge i file e risponde alle domande senza apportare
    modifiche.
```

```
``txt claude (plan mode) theme={null}
read /src/auth and understand how we handle sessions and login.
```

```
also look at how we manage environment variables for secrets.
```
```

```
<Step title="Pianifica">
```

Chiedi a Claude di creare un piano di implementazione dettagliato.

```
```txt
claude (plan mode) theme={null}
I want to add Google OAuth. What files need to change?
What's the session flow? Create a plan.
```
```

Premi `Ctrl+G` per aprire il piano nel vostro editor di testo per la modifica diretta prima che Claude proceda.

```
<Step title="Implementa">
```

Esci da Plan Mode e lascia che Claude codifichi, verificando rispetto al suo piano.

```
``txt claude (default mode) theme={null}
implement the OAuth flow from your plan. write tests for the
callback handler, run the test suite and fix any failures.
``
```

&lt;/Step&gt;

```
<Step title="Commit">
```

Chiedi a Claude di eseguire il commit con un messaggio descrittivo e creare una PR.

```
```txt claude (default mode) theme={null}
commit with a descriptive message and open a PR
```
```

&lt;/Steps&gt;

<Callout>

Plan Mode è utile, ma aggiunge anche overhead.

Per attività in cui l'ambito è chiaro e la correzione è piccola (come correggere un errore di battitura, aggiungere una riga di log o rinominare una variabile) chiedete a Claude di farlo direttamente.

La pianificazione è più utile quando siete incerti sull'approccio, quando il cambiamento modifica più file o quando non siete familiari con il codice da modificare. Se potete descrivere il diff in una frase, saltate il piano.

\*\*\*

## provide-specific-context-in-your-prompts

Fornite contesto specifico nei vostri prompt

## 

<Tip>

Più precise sono le vostre istruzioni, meno correzioni vi serviranno.

&lt;/Tip&gt;

Claude può dedurre l'intento, ma non può leggere la mente. Fate riferimento a file specifici, menzionate i vincoli e indicate i modelli di esempio.

| Strategia

Prima

| Dopo

| \_\_\_\_\_  
| \_\_\_\_\_ | \_\_\_\_\_ | \_\_\_\_\_  
| \_\_\_\_\_  
| \_\_\_\_\_  
| \_\_\_\_\_

| \*\*Definire l'ambito del compito.\*\* Specificate quale file, quale scenario e le

```

preferenze di test. | *"aggiungere test per foo.py"* | *"scrivere un
test per foo.py che copra il caso limite in cui l'utente è disconnesso. evitare i mock."*
|
| **Puntare alle fonti.** Indirizzate Claude alla fonte che può rispondere a una domanda.
| *"perché ExecutionFactory ha un'API così strana?"* | *"guardare la cronologia git di
ExecutionFactory e riassumere come la sua API è arrivata a essere così"*
|
| **Fare riferimento ai modelli esistenti.** Puntate Claude ai modelli nel vostro
codebase. | *"aggiungere un widget calendario"* | *"guardare come
i widget esistenti sono implementati nella home page per comprendere i modelli.
HotDogWidget.php è un buon esempio. seguire il modello per implementare un nuovo widget
calendario che consenta all'utente di selezionare un mese e paginare avanti/indietro per
scegliere un anno. costruire da zero senza librerie diverse da quelle già utilizzate nel
codebase."* |
| *"Descrivere il sintomo."* Fornite il sintomo, la probabile posizione e come appare
"risolto". | *"correggere il bug di login"* | *"gli utenti
segnalano che il login fallisce dopo il timeout della sessione. controllare il flusso di
autenticazione in src/auth/, in particolare l'aggiornamento del token. scrivere un test
fallito che riproduce il problema, quindi correggerlo"*
|

```

I prompt vaghi possono essere utili quando state esplorando e potete permettervi di correggere il corso. Un prompt come `"cosa migliorereste in questo file?"` può evidenziare cose a cui non avreste pensato di chiedere.

```

<h3 id="provide-rich-content">
 Fornire contenuti ricchi
</h3>

```

```

<Tip>
 Utilizzate `@` per fare riferimento ai file, incollate screenshot/immagini o inviate i
 dati direttamente.
</Tip>

```

Potete fornire dati ricchi a Claude in diversi modi:

```

* **Fare riferimento ai file con `@`** invece di descrivere dove vive il codice. Claude
legge il file prima di rispondere.
* **Incollare le immagini direttamente**.
```

Copiate/incollate o trascinate le immagini nel prompt.

```

* **Fornire URL** per la documentazione e i riferimenti API. Utilizzate `/permissions`
per aggiungere alla whitelist i domini utilizzati di frequente.
* **Inviare i dati tramite pipe** eseguendo `cat error.log | claude` per inviare il
contenuto del file direttamente.
* **Lasciare che Claude recuperi quello di cui ha bisogno**.
```

Dite a Claude di estrarre il contesto stesso utilizzando comandi Bash, strumenti MCP o leggendo i file.

```

```

```

<h2 id="configure-your-environment">
 Configurare il vostro ambiente
</h2>

```

Alcuni passaggi di configurazione rendono Claude Code significativamente più efficace in tutte le vostre sessioni. Per una panoramica completa delle funzionalità dell'estensione e di quando utilizzare ciascuna, consultate [Extend Claude Code](/it/features-overview).

```

<h3 id="write-an-effective-claude-md">
 Scrivete un CLAUDE.md efficace
</h3>

```

```

<Tip>
 Eseguite `/init` per generare un file CLAUDE.md iniziale basato sulla struttura del
 vostro progetto attuale, quindi affinate nel tempo.
</Tip>

```

CLAUDE.md è un file speciale che Claude legge all'inizio di ogni conversazione. Includete comandi Bash, stile del codice e regole del flusso di lavoro. Questo fornisce a Claude un contesto persistente che non può dedurre dal solo codice.

Non esiste un formato obbligatorio per i file CLAUDE.md, ma manteneteli brevi e leggibili. Ad esempio:

```

``markdown CLAUDE.md theme={null}
Code style
- Use ES modules (import/export) syntax, not CommonJS (require)
- Destructure imports when possible (eg. import { foo } from 'bar')

Workflow
- Be sure to typecheck when you're done making a series of code changes
- Prefer running single tests, and not the whole test suite, for performance
``

```

CLAUDE.md viene caricato ogni sessione, quindi includete solo le cose che si applicano ampiamente. Per la conoscenza del dominio o i flussi di lavoro che sono rilevanti solo a volte, utilizzate [skills]/(it/skills) invece. Claude li carica su richiesta senza gonfiare ogni conversazione.

Mantenetelo conciso. Per ogni riga, chiedetevi: `"Rimuovere questo causerebbe a Claude di fare errori?"` Se no, tagliatelo. I file CLAUDE.md gonfi causano a Claude di ignorare le vostre istruzioni effettive!

| ✔ Includere                                                                                                        | ✖ Escludere        |
|--------------------------------------------------------------------------------------------------------------------|--------------------|
| Comandi Bash che Claude non può indovinare<br>Claude possa capire leggendo il codice                               | Qualsiasi cosa     |
| Regole di stile del codice che differiscono dai valori predefiniti<br>linguistiche standard che Claude già conosce | Convenzioni        |
| Istruzioni di test e runner di test preferiti<br>dettagliata (collegare alla documentazione invece)                | Documentazione API |
| Etichetta del repository (denominazione dei rami, convenzioni PR)<br>cambiano frequentemente                       | Informazioni che   |
| Decisioni architetturiche specifiche del vostro progetto<br>o tutorial                                             | Spiegazioni lunghe |
| Stranezze dell'ambiente di sviluppo (variabili env richieste)<br>per file del codebase                             | Descrizioni file   |
| Gotcha comuni o comportamenti non ovvi<br>evidenti come "scrivere codice pulito"                                   | Pratiche auto-     |

Se Claude continua a fare qualcosa che non volete nonostante abbia una regola contro di essa, il file è probabilmente troppo lungo e la regola si sta perdendo. Se Claude vi fa domande che sono risposte in CLAUDE.md, la formulazione potrebbe essere ambigua. Trattate CLAUDE.md come codice: rivederlo quando le cose vanno male, potarlo regolarmente e testate i cambiamenti osservando se il comportamento di Claude effettivamente cambia.

Potete sintonizzare le istruzioni aggiungendo enfasi (ad es. "IMPORTANTE" o "DOVETE") per migliorare l'aderenza. Controllate CLAUDE.md in git in modo che il vostro team possa contribuire. Il file aumenta di valore nel tempo.

I file CLAUDE.md possono importare file aggiuntivi utilizzando la sintassi ``@path/to/import``:

```

``markdown CLAUDE.md theme={null}
See @README.md for project overview and @package.json for available npm commands.

Additional Instructions
- Git workflow: @docs/git-instructions.md
- Personal overrides: @~/ .claude/my-project-instructions.md
``

```

Potete posizionare i file CLAUDE.md in diversi percorsi:

\* \*\*Cartella home (`~/claude/CLAUDE.md`)\*\*: si applica a tutte le sessioni Claude

- \* **Radice del progetto** (`./CLAUDE.md`): controllare in git per condividere con il vostro team
- \* **Radice del progetto** (`./CLAUDE.local.md`): note personali specifiche del progetto; aggiungete questo file al vostro `.gitignore` in modo che non sia condiviso con il vostro team
- \* **Directory padre**: utile per monorepo dove sia `root/CLAUDE.md` che `root/foo/CLAUDE.md` vengono estratti automaticamente
- \* **Directory figlie**: Claude estrae i file `CLAUDE.md` figlio su richiesta quando lavora con i file in quelle directory

### configure-permissions

Configurate i permessi

#### Tip

Utilizzate `[auto mode](/it/permission-modes#eliminate-prompts-with-auto-mode)` per lasciare che un classificatore gestisca le approvazioni, `/permissions` per aggiungere alla whitelist comandi specifici, o `/sandbox` per l'isolamento a livello di sistema operativo. Ciascuno riduce le interruzioni mantenendovi il controllo.

Per impostazione predefinita, Claude Code richiede il permesso per azioni che potrebbero modificare il vostro sistema: scritture di file, comandi Bash, strumenti MCP, ecc. Questo è sicuro ma tedioso. Dopo la decima approvazione non state davvero revisionando più, state solo cliccando. Ci sono tre modi per ridurre queste interruzioni:

- \* **Auto mode**: un modello classificatore separato esamina i comandi e blocca solo quello che sembra rischioso: escalation di ambito, infrastruttura sconosciuta o azioni guidate da contenuti ostili. Migliore quando fidate della direzione generale di un'attività ma non volete cliccare attraverso ogni passaggio
- \* **Whitelist di permessi**: consentire strumenti specifici che sapete essere sicuri, come `npm run lint` o `git commit`
- \* **Sandboxing**: abilitare l'isolamento a livello di sistema operativo che limita l'accesso al filesystem e alla rete, consentendo a Claude di lavorare più liberamente all'interno di confini definiti

Leggete di più su `[permission modes](/it/permission-modes)`, `[permission rules](/it/permissions)` e `[sandboxing](/it/sandboxing)`.

### use-cli-tools

Utilizzate gli strumenti CLI

#### Tip

Dite a Claude Code di utilizzare strumenti CLI come `gh`, `aws`, `gcloud` e `sentry-cli` quando interagite con servizi esterni.

Gli strumenti CLI sono il modo più efficiente in termini di contesto per interagire con servizi esterni. Se utilizzate GitHub, installate la CLI `gh`. Claude sa come usarla per creare problemi, aprire pull request e leggere commenti. Senza `gh`, Claude può comunque utilizzare l'API GitHub, ma le richieste non autenticate spesso raggiungono i limiti di velocità.

Claude è anche efficace nell'imparare strumenti CLI che non conosce già. Provate prompt come `'Use 'foo-cli-tool --help' to learn about foo tool, then use it to solve A, B, C.'`

### connect-mcp-servers

Connettete i server MCP

#### Tip

Eseguite `claude mcp add` per connettere strumenti esterni come Notion, Figma o il vostro database.

Con i `[server MCP](/it/mcp)`, potete chiedere a Claude di implementare funzionalità da tracker di problemi, interrogare database, analizzare dati di monitoraggio, integrare design da Figma e automatizzare flussi di lavoro.

```
<h3 id="set-up-hooks">
 Configurare gli hook
</h3>
```

```
<Tip>
 Utilizzate gli hook per azioni che devono accadere ogni volta senza eccezioni.
</Tip>
```

Gli [hook](/it/hooks-guide) eseguono script automaticamente in punti specifici del flusso di lavoro di Claude. A differenza delle istruzioni CLAUDE.md che sono consultive, gli hook sono deterministici e garantiscono che l'azione accada.

Claude può scrivere hook per voi. Provate prompt come `*"Write a hook that runs eslint after every file edit"*` o `*"Write a hook that blocks writes to the migrations folder."*` Modificate ``.claude/settings.json`` direttamente per configurare gli hook manualmente e eseguite ``.hooks`` per sfogliare quello che è configurato.

```
<h3 id="create-skills">
 Createte skill
</h3>
```

```
<Tip>
 Create file `SKILL.md` in `.claude/skills/` per fornire a Claude conoscenza del dominio
 e flussi di lavoro riutilizzabili.
</Tip>
```

Le [skill](/it/skills) estendono la conoscenza di Claude con informazioni specifiche del vostro progetto, team o dominio. Claude le applica automaticamente quando rilevanti, o potete invocarle direttamente con ``.skill-name``.

Create una skill aggiungendo una directory con un ``.SKILL.md`` a ``.claude/skills/``:

```
```markdown .claude/skills/api-conventions/SKILL.md theme={null}
---
name: api-conventions
description: REST API design conventions for our services
---
# API Conventions
- Use kebab-case for URL paths
- Use camelCase for JSON properties
- Always include pagination for list endpoints
- Version APIs in the URL path (/v1/, /v2/)
```
```

Le skill possono anche definire flussi di lavoro ripetibili che invocano direttamente:

```
```markdown .claude/skills/fix-issue/SKILL.md theme={null}
---
name: fix-issue
description: Fix a GitHub issue
disable-model-invocation: true
---
Analyze and fix the GitHub issue: $ARGUMENTS.
```

1. Use ``.gh issue view`` to get the issue details
2. Understand the problem described in the issue
3. Search the codebase for relevant files
4. Implement the necessary changes to fix the issue
5. Write and run tests to verify the fix
6. Ensure code passes linting and type checking
7. Create a descriptive commit message
8. Push and create a PR

Eseguite ``.fix-issue 1234`` per invocarlo. Utilizzate ``.disable-model-invocation: true`` per flussi di lavoro con effetti collaterali che volete attivare manualmente.

```
<h3 id="create-custom-subagents">
```

Create subagent personalizzati

<Tip>

Definite assistenti specializzati in ``.claude/agents/`` che Claude può delegare per attività isolate.

</Tip>

I `[subagent](/it/sub-agents)` vengono eseguiti nel loro contesto con il loro set di strumenti consentiti. Sono utili per attività che leggono molti file o necessitano di focus specializzato senza ingombrare la vostra conversazione principale.

```
```markdown .claude/agents/security-reviewer.md theme={null}
```

```

```

```
name: security-reviewer
description: Reviews code for security vulnerabilities
tools: Read, Grep, Glob, Bash
model: opus

```

You are a senior security engineer. Review code for:

- Injection vulnerabilities (SQL, XSS, command injection)
- Authentication and authorization flaws
- Secrets or credentials in code
- Insecure data handling

Provide specific line references and suggested fixes.

```
```
```

Dite a Claude di utilizzare i subagent esplicitamente: `*"Use a subagent to review this code for security issues."*`

<h3 id="install-plugins">

Installate i plugin

</h3>

<Tip>

Eseguite ``/plugin`` per sfogliare il marketplace. I plugin aggiungono skill, strumenti e integrazioni senza configurazione.

</Tip>

I `[plugin](/it/plugins)` raggruppano skill, hook, subagent e server MCP in una singola unità installabile dalla comunità e da Anthropic. Se lavorate con un linguaggio tipizzato, installate un `[plugin di code intelligence](/it/discover-plugins#code-intelligence)` per fornire a Claude la navigazione precisa dei simboli e il rilevamento automatico degli errori dopo le modifiche.

Per una guida sulla scelta tra skill, subagent, hook e MCP, consultate `[Extend Claude Code](/it/features-overview#match-features-to-your-goal)`.

<h2 id="communicate-effectively">

Comunicare efficacemente

</h2>

Il modo in cui comunicate con Claude Code ha un impatto significativo sulla qualità dei risultati.

<h3 id="ask-codebase-questions">

Fate domande sul codebase

</h3>

<Tip>

Fate a Claude domande che fareste a un ingegnere senior.

</Tip>

Quando vi onboarding a un nuovo codebase, utilizzate Claude Code per l'apprendimento e l'esplorazione. Potete fare a Claude lo stesso tipo di domande che fareste a un altro ingegnere:

- * Come funziona il logging?
- * Come faccio a creare un nuovo endpoint API?
- * Cosa fa ``async move { ... }`` sulla riga 134 di ``foo.rs``?
- * Quali casi limite gestisce ``CustomerOnboardingFlowImpl``?
- * Perché questo codice chiama ``foo()`` invece di ``bar()`` sulla riga 333?

Utilizzare Claude Code in questo modo è un flusso di lavoro di onboarding efficace, migliorando il tempo di ramp-up e riducendo il carico su altri ingegneri. Non è richiesto alcun prompt speciale: fate le domande direttamente.

```
<h3 id="let-claude-interview-you">
  Lasciate che Claude vi intervisti
</h3>
```

```
<Tip>
  Per funzionalità più grandi, fate intervistare Claude prima. Iniziate con un prompt
  minimo e chiedete a Claude di intervistarvi utilizzando lo strumento `AskUserQuestion`.
</Tip>
```

Claude fa domande su cose che potreste non aver ancora considerato, inclusa l'implementazione tecnica, l'interfaccia utente/esperienza utente, i casi limite e i compromessi.

```
```text theme={null}
I want to build [brief description]. Interview me in detail using the AskUserQuestion
tool.
```

Ask about technical implementation, UI/UX, edge cases, concerns, and tradeoffs. Don't ask obvious questions, dig into the hard parts I might not have considered.

Keep interviewing until we've covered everything, then write a complete spec to SPEC.md.

Una volta completata la specifica, avviate una nuova sessione per eseguirla. La nuova sessione ha un contesto pulito focalizzato interamente sull'implementazione e avete una specifica scritta a cui fare riferimento.

Le specifiche più utili sono autonome: nominano i file e le interfacce coinvolte, dichiarano cosa è fuori ambito e terminano con un passaggio di verifica end-to-end che prova che la funzionalità funziona. Il tempo speso per rendere la specifica precisa ripaga più del tempo speso a guardare l'implementazione.

\*\*\*

```
<h2 id="manage-your-session">
 Gestite la vostra sessione
</h2>
```

Le conversazioni sono persistenti e reversibili. Utilizzate questo a vostro vantaggio!

```
<h3 id="course-correct-early-and-often">
 Correggete il corso presto e spesso
</h3>
```

```
<Tip>
 Correggete Claude non appena notate che sta andando fuori strada.
</Tip>
```

I migliori risultati provengono da cicli di feedback stretti. Sebbene Claude occasionalmente risolva i problemi perfettamente al primo tentativo, correggerlo rapidamente generalmente produce soluzioni migliori più velocemente.

- \* `**`Esc`**`: fermate Claude a metà azione con il tasto ``Esc``. Il contesto viene preservato, quindi potete reindirizzare.
- \* `**`Esc + Esc`` o ``/rewind`**`: premete ``Esc`` due volte o eseguite ``/rewind`` per aprire il menu di rewind e ripristinare la conversazione e lo stato del codice precedenti, o riassumere da un messaggio selezionato.
- \* `**`Undo that`**`: fate in modo che Claude ripristini le sue modifiche.

\* `*/clear*`: ripristinate il contesto tra attività non correlate. Le sessioni lunghe con contesto irrilevante possono ridurre le prestazioni.

Se avete corretto Claude più di due volte sullo stesso problema in una sessione, il contesto è ingombro di approcci falliti. Eseguite `*/clear*` e iniziate di nuovo con un prompt più specifico che incorpori quello che avete imparato. Una sessione pulita con un prompt migliore quasi sempre supera una sessione lunga con correzioni accumulate.

`<h3 id="manage-context-aggressively">`  
 Gestite il contesto aggressivamente  
`</h3>`

`<Tip>`  
 Eseguite `*/clear*` tra attività non correlate per ripristinare il contesto.  
`</Tip>`

Claude Code compatta automaticamente la cronologia della conversazione quando vi avvicinate ai limiti del contesto, il che preserva il codice e le decisioni importanti mentre libera spazio.

Durante le sessioni lunghe, la finestra di contesto di Claude può riempirsi di conversazioni irrilevanti, contenuti di file e comandi. Questo può ridurre le prestazioni e talvolta distrarre Claude.

- \* Utilizzate `*/clear*` frequentemente tra le attività per ripristinare completamente la finestra di contesto
- \* Quando la compattazione automatica si attiva, Claude riassume quello che conta di più, inclusi i modelli di codice, gli stati dei file e le decisioni chiave
- \* Per un maggiore controllo, eseguite `*/compact <instructions>*`, come `*/compact Focus on the API changes*`
- \* Per compattare solo parte della conversazione, utilizzate `Esc + Esc` o `*/rewind*`, selezionate un checkpoint di messaggio e scegliete `**Summarize from here**` o `**Summarize up to here**`. Il primo condensa i messaggi da quel punto in poi mantenendo il contesto precedente intatto; il secondo condensa i messaggi precedenti mantenendo quelli recenti in pieno. Consultate [Restore vs. summarize](/it/checkpointing#restore-vs-summarize).
- \* Personalizzate il comportamento di compattazione in CLAUDE.md con istruzioni come `"When compacting, always preserve the full list of modified files and any test commands"` per assicurare che il contesto critico sopravviva alla riassunzione
- \* Per domande rapide che non devono rimanere nel contesto, utilizzate `[*/btw*]` (/it/interactive-mode#side-questions-with-%2Fbtw). La risposta appare in un overlay dismissibile e non entra mai nella cronologia della conversazione, quindi potete controllare un dettaglio senza far crescere il contesto.

`<h3 id="use-subagents-for-investigation">`  
 Utilizzate i subagent per l'investigazione  
`</h3>`

`<Tip>`  
 Delegate la ricerca con `"use subagents to investigate X"`. Esplorano in un contesto separato, mantenendo la vostra conversazione principale pulita per l'implementazione.  
`</Tip>`

Poiché il contesto è il vostro vincolo fondamentale, i subagent sono uno degli strumenti più potenti disponibili. Quando Claude ricerca un codebase legge molti file, tutti i quali consumano il vostro contesto. I subagent vengono eseguiti in finestre di contesto separate e riportano riassunti:

```
```text theme={null}
Use subagents to investigate how our authentication system handles token
refresh, and whether we have any existing OAuth utilities I should reuse.
```
```

Il subagent esplora il codebase, legge i file rilevanti e riporta i risultati, il tutto senza ingombrare la vostra conversazione principale.

Potete anche utilizzare i subagent per la verifica dopo che Claude implementa qualcosa:

```
```text theme={null}
use a subagent to review this code for edge cases
```
```

### `<h3 id="rewind-with-checkpoints">`

Riavvolgete con i checkpoint  
`</h3>`

#### `<Tip>`

Ogni prompt che inviate crea un checkpoint. Potete ripristinare la conversazione, il codice o entrambi a qualsiasi checkpoint precedente.

#### `</Tip>`

Claude crea automaticamente snapshot dei file prima di ogni modifica in modo che un checkpoint possa ripristinarli. Premete il doppio tasto Escape o eseguite ``/rewind`` per aprire il menu di rewind. Potete ripristinare solo la conversazione, ripristinare solo il codice, ripristinare entrambi o riassumere da un messaggio selezionato. Consultate [\[Checkpointing\]\(/it/checkpointing\)](#) per i dettagli.

Invece di pianificare attentamente ogni mossa, potete dire a Claude di provare qualcosa di rischioso. Se non funziona, riavvolgete e provate un approccio diverso. I checkpoint persistono tra le sessioni, quindi potete chiudere il vostro terminale e comunque riavvolgere in seguito.

#### `<Warning>`

I checkpoint tracciano solo le modifiche apportate *\*da Claude\**, non i processi esterni. Questo non è un sostituto per git.

#### `</Warning>`

### `<h3 id="resume-conversations">`

Riprendete le conversazioni  
`</h3>`

#### `<Tip>`

Nominate le sessioni con ``/rename`` e trattate le come rami: ogni flusso di lavoro ottiene il suo contesto persistente.

#### `</Tip>`

Claude Code salva le conversazioni localmente, quindi quando un'attività si estende su più sessioni non dovete rispiegare il contesto. Eseguite ``claude --continue`` per riprendere la sessione più recente, o ``claude --resume`` per scegliere da un elenco. Assegnate alle sessioni nomi descrittivi come ``oauth-migration`` in modo da poterle trovare in seguito. Consultate [\[Manage sessions\]\(/it/sessions\)](#) per l'insieme completo di controlli di ripresa, ramo e denominazione.

\*\*\*

## `<h2 id="automate-and-scale">`

Automatizzate e ridimensionate  
`</h2>`

Una volta che siete efficaci con un Claude, moltiplicate il vostro output con sessioni parallele, modalità non interattiva e modelli di fan-out.

Tutto finora presuppone un umano, un Claude e una conversazione. Ma Claude Code si ridimensiona orizzontalmente. Le tecniche in questa sezione mostrano come potete fare di più.

### `<h3 id="run-non-interactive-mode">`

Eseguite la modalità non interattiva  
`</h3>`

#### `<Tip>`

Utilizzate ``claude -p "prompt"`` in CI, pre-commit hooks o script. Aggiungete ``--output-format stream-json --verbose`` per l'output JSON in streaming.

#### `</Tip>`

Con ``claude -p "your prompt"``, potete eseguire Claude in modo non interattivo, senza una sessione. [\[La modalità non interattiva\]\(/it/headless\)](#) è il modo in cui integrate Claude nelle pipeline CI, pre-commit hooks o qualsiasi flusso di lavoro automatizzato. I formati di output vi permettono di analizzare i risultati a livello di programmazione: testo

semplice, JSON o JSON in streaming.

```
```bash theme={null}
# One-off queries
claude -p "Explain what this project does"

# Structured output for scripts
claude -p "List all API endpoints" --output-format json

# Streaming for real-time processing
claude -p "Analyze this log file" --output-format stream-json --verbose
```
```

<h3 id="run-multiple-claude-sessions">  
Eseguite più sessioni Claude  
</h3>

<Tip>

Eseguite più sessioni Claude in parallelo per accelerare lo sviluppo, eseguire esperimenti isolati o avviare flussi di lavoro complessi.  
</Tip>

Scegliete l'approccio parallelo che si adatta a quanto coordinamento volete fare voi stessi:

- \* [Worktrees](/it/worktrees): eseguite sessioni CLI separate in checkout git isolati in modo che le modifiche non si scontrino
- \* [App desktop](/it/desktop#work-in-parallel-with-sessions): gestite più sessioni locali visivamente, ciascuna nel suo worktree
- \* [Claude Code sul web](/it/claude-code-on-the-web): eseguite sessioni su infrastruttura cloud gestita da Anthropic in VM isolate
- \* [Team di agenti](/it/agent-teams): coordinamento automatizzato di più sessioni con attività condivise, messaggistica e un team lead

Oltre a parallelizzare il lavoro, più sessioni consentono flussi di lavoro focalizzati sulla qualità. Un contesto fresco migliora la revisione del codice poiché Claude non sarà distorto verso il codice che ha appena scritto.

Ad esempio, utilizzate un modello Writer/Reviewer:

| Sessione A (Writer)<br>(Reviewer)                                                                                                                            | Sessione B  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| -----                                                                                                                                                        | -----       |
| -----                                                                                                                                                        |             |
| `Implement a rate limiter for our API endpoints`                                                                                                             |             |
|                                                                                                                                                              | `Review the |
| rate limiter implementation in @src/middleware/rateLimiter.ts. Look for edge cases, race conditions, and consistency with our existing middleware patterns.` |             |
| `Here's the review feedback: [Session B output]. Address these issues.`                                                                                      |             |
|                                                                                                                                                              |             |

Potete fare qualcosa di simile con i test: fate scrivere i test a un Claude, poi a un altro scrivere il codice per passarli.

<h3 id="fan-out-across-files">  
Fan out tra i file  
</h3>

<Tip>

Eseguite un ciclo attraverso le attività chiamando `claude -p` per ciascuna. Utilizzate `--allowedTools` per limitare i permessi per le operazioni batch.  
</Tip>

Per migrazioni o analisi su larga scala, potete distribuire il lavoro tra molte invocazioni parallele di Claude:

```

<Steps>
 <Step title="Generare un elenco di attività">
 Fate in modo che Claude elenchi tutti i file che devono essere migrati (ad es. `list
 all 2,000 Python files that need migrating`)
 </Step>

 <Step title="Scrivere uno script per eseguire un ciclo attraverso l'elenco">
    ```bash theme={null}
    for file in $(cat files.txt); do
      claude -p "Migrate $file from React to Vue. Return OK or FAIL." \
        --allowedTools "Edit,Bash(git commit *)"
    done
    ```
 </Step>

 <Step title="Testare su pochi file, quindi eseguire su larga scala">
 Affinate il vostro prompt in base a quello che va male con i primi 2-3 file, quindi
 eseguite sull'intero set. Il flag `--allowedTools` limita quello che Claude può fare, il
 che è importante quando state eseguendo senza supervisione.
 </Step>
</Steps>

```

Potete anche integrare Claude nelle pipeline di elaborazione/dati esistenti:

```

```bash theme={null}
claude -p "<your prompt>" --output-format json | your_command
```

```

Utilizzate `--verbose` per il debug durante lo sviluppo e spegnetelo in produzione.

```

<h3 id="run-autonomously-with-auto-mode">
 Eseguite autonomamente con auto mode
</h3>

```

Per l'esecuzione ininterrotta con controlli di sicurezza in background, utilizzate [auto mode](/it/permission-modes#eliminate-prompts-with-auto-mode). Un modello classificatore esamina i comandi prima che vengano eseguiti, bloccando l'escalation di ambito, l'infrastruttura sconosciuta e le azioni guidate da contenuti ostili mentre consente al lavoro di routine di procedere senza prompt.

```

```bash theme={null}
claude --permission-mode auto -p "fix all lint errors"
```

```

Per esecuzioni non interattive con il flag `-p`, auto mode interrompe se il classificatore blocca ripetutamente le azioni, poiché non c'è un utente a cui ricorrere. Consultate [when auto mode falls back](/it/permission-modes#when-auto-mode-falls-back) per le soglie.

```

<h3 id="add-an-adversarial-review-step">
 Aggiungete un passaggio di revisione avversariale
</h3>

```

```

<Tip>
 Prima di considerare un'attività come completata, fate in modo che un subagent riveda
 il diff in un contesto fresco e segnali le lacune.
</Tip>

```

Più a lungo Claude lavora senza supervisione, più un controllo indipendente è importante prima di contare il lavoro come completato. Un revisore che esegue in un contesto [subagent](/it/sub-agents) fresco vede solo il diff e i criteri che gli date, non il ragionamento che ha prodotto il cambiamento, quindi valuta il risultato secondo i suoi stessi termini.

Per un controllo di correttezza, eseguite la skill [code-review](/it/commands) in bundle, che rivede il diff corrente per bug in un subagent fresco e restituisce i risultati alla sessione. Per controllare il diff rispetto al vostro piano, scrivete il prompt di revisione voi stessi. Nominate il lavoro da controllare, il piano da controllare e cosa conta come una lacuna:

```
``text theme={null}
```

```
Use a subagent to review the rate limiter diff against PLAN.md. Check that
every requirement is implemented, the listed edge cases have tests, and
nothing outside the task's scope changed. Report gaps, not style preferences.
````
```

Poiché il revisore esegue come subagent, la sessione di implementazione riceve le lacune direttamente e può correggerle e riesaminarle senza che voi copiate i risultati tra le finestre. Per esecuzioni autonome più lunghe, un [agent team](/it/agent-teams) può mantenere questo ciclo in corso su molte attività mentre voi controllate i risultati registrati.

<Callout>

Un revisore incaricato di trovare lacune di solito ne segnala alcune, anche quando il lavoro è solido, perché è quello che gli è stato chiesto di fare. Inseguire ogni risultato porta a un'over-engineering: strati di astrazione extra, codice difensivo e test per casi che non possono accadere. Dite al revisore di segnalare solo le lacune che influiscono sulla correttezza o sui requisiti dichiarati, e trattate il resto come facoltativo.

</Callout>

<h2 id="avoid-common-failure-patterns">

Evitate i modelli di fallimento comuni
</h2>

Questi sono errori comuni. Riconoscerli presto fa risparmiare tempo:

* **La sessione del lavandino della cucina.** Iniziate con un'attività, poi chiedete a Claude qualcosa di non correlato, poi tornate alla prima attività. Il contesto è pieno di informazioni irrilevanti.

> **Correzione**:

* **Correggere più volte.** Claude fa qualcosa di sbagliato, lo correggete, è ancora sbagliato, lo correggete di nuovo. Il contesto è inquinato da approcci falliti.

> **Correzione**:

* **Il CLAUDE.md eccessivamente specificato.** Se il vostro CLAUDE.md è troppo lungo, Claude ignora metà di esso perché le regole importanti si perdono nel rumore.

> **Correzione**:

* **Il divario tra fiducia e verifica.** Claude produce un'implementazione plausibile che non gestisce i casi limite.

> **Correzione**:

* **L'esplorazione infinita.** Chiedete a Claude di "investigare" qualcosa senza limitarlo. Claude legge centinaia di file, riempiendo il contesto.

> **Correzione**:

<h2 id="develop-your-intuition">

Sviluppate la vostra intuizione
</h2>

I modelli in questa guida non sono scolpiti nella pietra. Sono punti di partenza che funzionano bene in generale, ma potrebbero non essere ottimali per ogni situazione.

A volte **dovreste** lasciare che il contesto si accumuli perché siete immersi in un problema complesso e la cronologia è preziosa. A volte dovreste saltare la pianificazione e lasciare che Claude lo capisca perché l'attività è esplorativa. A volte un prompt vago è esattamente giusto perché volete vedere come Claude interpreta il problema prima di limitarlo.

Prestate attenzione a quello che funziona. Quando Claude produce un output eccellente, notate quello che avete fatto: la struttura del prompt, il contesto che avete fornito, la modalità in cui siete stati. Quando Claude fatica, chiedetevi perché. Il contesto era

troppo rumoroso? Il prompt troppo vago? L'attività troppo grande per un passaggio?

Nel tempo, svilupperete un'intuizione che nessuna guida può catturare. Saprete quando essere specifici e quando essere aperti, quando pianificare e quando esplorare, quando cancellare il contesto e quando lasciarlo accumulare.

<h2 id="related-resources">
 Risorse correlate
</h2>

- * [How Claude Code works](/it/how-claude-code-works): il ciclo agenziale, gli strumenti e la gestione del contesto
- * [Extend Claude Code](/it/features-overview): skill, hook, MCP, subagent e plugin
- * [Common workflows](/it/common-workflows): ricette passo dopo passo per debug, test, PR e altro
- * [CLAUDE.md](/it/memory): archiviare le convenzioni del progetto e il contesto persistente